# Pet Facial Expression Classification using Deep Learning

## Objective of the Analysis

The primary goal of this analysis is to develop a Deep Learning model capable of classifying pet facial expressions into four categories: Angry, Happy, Sad, and Other.

This classification can have multiple applications, including:

- Animal Welfare: Assessing pet emotions for better care.
- Pet Marketing: Understanding animal emotions for targeted pet product advertisements.
- Veterinary Assistance: Supporting vets in diagnosing pet emotional distress.

We will be using Convolutional Neural Networks (CNNs) and Transfer Learning to analyze pet facial expressions.

## Dataset Description

The dataset used is the Pets Facial Expression Dataset from Kaggle. It consists of images of pets categorized into four facial expression classes:

- Angry
- Happy
- Sad
- Other

The dataset is structured into three main directories:

📁 MasterFolder/

├── 📁 train/ (Training images)

├── 📁 valid/ (Validation images)

├── 📁 test/ (Test images)

Each folder contains subfolders for each expression category.

Key Features:

- Input: Pet images of different breeds and species.
- Output: The expression category.
- Data Type: Image dataset.
- Data Preprocessing: Includes resizing, normalization, and data augmentation.

In [8]:

```python
import os

os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'  # Contournement du problème OpenMP

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np
import time
from tqdm import tqdm

# Optimisation pour le GPU lorsque la taille des entrées reste constante
torch.backends.cudnn.benchmark = True

# Vous pouvez exécuter cette cellule seule pour charger les modules et activer les optimisation
```

## Data Preprocessing & Exploration

### 3.1 Preprocessing Techniques

To prepare the dataset for training, the following transformations are applied:

- Resizing: All images are resized to 224x224 pixels.
- Normalization: The pixel values are normalized using ImageNet standards (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
- Data Augmentation (for the improved CNN model):
- Random horizontal flipping.
- Random rotation of up to ±10 degrees.

### 3.2 Loading Data

We use torchvision.datasets.ImageFolder to load images and create DataLoaders for:

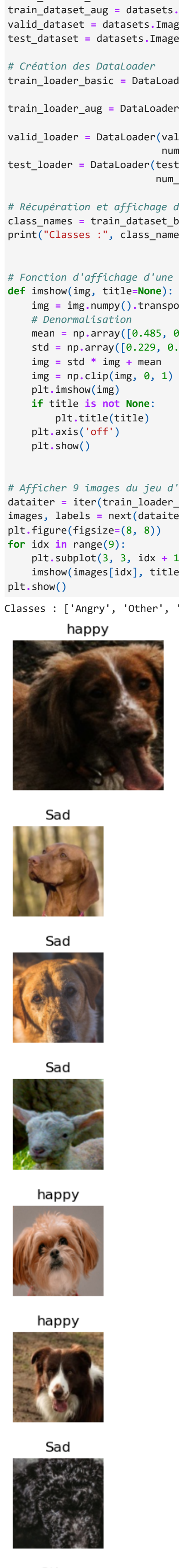
- Training (Basic and Augmented)
- Validation
- Testing

We also visualize a few sample images to confirm the dataset is correctly loaded.

In [9]:

```python
# Définition des chemins
base_dir = "./data/MasterFolder"
train_dir = os.path.join(base_dir, "train")
valid_dir = os.path.join(base_dir, "valid")
test_dir = os.path.join(base_dir, "test")

# Paramètres
batch_size = 32
img_size = (224, 224)
num_workers = 8  # Vous pouvez ajuster ce paramètre selon votre machine

# Définition des transformations
basic_transform = transforms.Compose([
    transforms.Resize(img_size),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

augmented_transform = transforms.Compose([
    transforms.Resize(img_size),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),  # rotation aléatoire jusqu'à ±10°
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# Chargement des datasets
train_dataset_basic = datasets.ImageFolder(train_dir, transform=basic_transform)
train_dataset_aug = datasets.ImageFolder(train_dir, transform=augmented_transform)
valid_dataset = datasets.ImageFolder(valid_dir, transform=basic_transform)
test_dataset = datasets.ImageFolder(test_dir, transform=basic_transform)

# Création des DataLoader
train_loader_basic = DataLoader(train_dataset_basic, batch_size=batch_size, shuffle=True,
                                num_workers=num_workers, pin_memory=True)
train_loader_aug = DataLoader(train_dataset_aug, batch_size=batch_size, shuffle=True,
                              num_workers=num_workers, pin_memory=True)
valid_loader = DataLoader(valid_dataset, batch_size=batch_size, shuffle=False,
                          num_workers=num_workers, pin_memory=True)
test_loader = DataLoader(test_dataset, batch_size=batch_size, shuffle=False,
                         num_workers=num_workers, pin_memory=True)

# Récupération et affichage des classes
class_names = train_dataset_basic.classes
print("Classes :", class_names)


# Fonction d'affichage d'une image
def imshow(img, title=None):
    img = img.numpy().transpose((1, 2, 0))
    # Denormalisation
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    img = std * img + mean
    img = np.clip(img, 0, 1)
    plt.imshow(img)
    if title is not None:
        plt.title(title)
    plt.axis('off')
    plt.show()


# Afficher 9 images du jeu d'entraînement basique
dataiter = iter(train_loader_basic)
images, labels = next(dataiter)
plt.figure(figsize=(8, 8))
for idx in range(9):
    plt.subplot(3, 3, idx + 1)
    imshow(images[idx], title=class_names[labels[idx]])
plt.show()
```

```
Classes : ['Angry', 'Other', 'Sad', 'happy']
```



# Deep Learning Models Implemented

We train three different Deep Learning models to compare their performances:

## Model 1: Basic CNN

- A simple 2-layer convolutional neural network with ReLU activation and max-pooling.
- Uses fully connected layers for classification.
- Serves as the baseline model.

## Model 2: Improved CNN

- Uses Batch Normalization and Dropout to improve regularization.
- Includes an additional convolutional layer for deeper feature extraction.
- Incorporates data augmentation to increase model robustness.

## Model 3: Transfer Learning (MobileNetV2)

- MobileNetV2, pretrained on ImageNet, is used for feature extraction.
- The classifier layer is modified to fit our dataset.
- The base layers of MobileNetV2 are frozen, only the classifier is trained.
- Each model is trained using Cross-Entropy Loss and Adam Optimizer.

In [10]:

```python
num_classes = 4


# Modèle 1 : CNN de base
class SimpleCNN(nn.Module):
    def __init__(self, num_classes=num_classes):
        super(SimpleCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),  # [B, 32, 224, 224]
            nn.ReLU(),
            nn.MaxPool2d(2, 2),  # [B, 32, 112, 112]
            nn.Conv2d(32, 64, kernel_size=3, padding=1),  # [B, 64, 112, 112]
            nn.ReLU(),
            nn.MaxPool2d(2, 2)  # [B, 64, 56, 56]
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(64 * 56 * 56, 128),
            nn.ReLU(),
            nn.Linear(128, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x


# Modèle 2 : CNN amélioré avec Data Augmentation, BatchNorm et Dropout
class ImprovedCNN(nn.Module):
    def __init__(self, num_classes=num_classes):
        super(ImprovedCNN, self).__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),  # [B, 32, 112, 112]

            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.ReLU(),
            nn.MaxPool2d(2, 2),  # [B, 64, 56, 56]

            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.BatchNorm2d(128),
            nn.ReLU(),
            nn.MaxPool2d(2, 2)  # [B, 128, 28, 28]
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 28 * 28, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes)
        )

    def forward(self, x):
        x = self.features(x)
        x = self.classifier(x)
        return x


# Modèle 3 : Transfert d'apprentissage avec MobileNetV2
class TransferMobileNet(nn.Module):
    def __init__(self, num_classes=num_classes):
        super(TransferMobileNet, self).__init__()
        self.base_model = models.mobilenet_v2(pretrained=True)
        # Geler la base
        for param in self.base_model.parameters():
            param.requires_grad = False
        in_features = self.base_model.last_channel  # généralement 1280
        self.base_model.classifier = nn.Sequential(
            nn.Dropout(0.3),
            nn.Linear(in_features, num_classes)
        )

    def forward(self, x):
        return self.base_model(x)
```

# Training Process & Hyperparameters

Each model is trained for 5 epochs using the following hyperparameters:

- Batch Size: 32
- Learning Rate: 0.001 for SimpleCNN, 0.0001 for others
- Optimizer: Adam
- Loss Function: CrossEntropyLoss

We use tqdm for progress tracking and visualize:

- Training Loss
- Validation Loss
- Training Accuracy
- Validation Accuracy

In [11]:

```python
def train_epoch(model, dataloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    running_corrects = 0
    total = 0
    # Boucle avec barre de chargement
    for inputs, labels in tqdm(dataloader, desc="Training", leave=False):
        inputs = inputs.to(device)
        labels = labels.to(device)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)
        _, preds = torch.max(outputs, 1)
        running_corrects += torch.sum(preds == labels.data)
        total += inputs.size(0)
    epoch_loss = running_loss / total
    epoch_acc = running_corrects.double() / total
    return epoch_loss, epoch_acc.item()


def evaluate_model(model, dataloader, criterion, device):
    model.eval()
    running_loss = 0.0
    running_corrects = 0
    total = 0
    with torch.no_grad():
        for inputs, labels in tqdm(dataloader, desc="Validation", leave=False):
            inputs = inputs.to(device)
            labels = labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            running_loss += loss.item() * inputs.size(0)
            _, preds = torch.max(outputs, 1)
            running_corrects += torch.sum(preds == labels.data)
            total += inputs.size(0)
    loss = running_loss / total
    acc = running_corrects.double() / total
    return loss, acc.item()


def train_model(model, train_loader, valid_loader, criterion, optimizer, device, num_epochs=5):
    best_acc = 0.0
    train_loss_history = []
    train_acc_history = []
    valid_loss_history = []
    valid_acc_history = []

    for epoch in range(num_epochs):
        start_time = time.time()
        print(f"Epoch {epoch + 1}/{num_epochs}")
        train_loss, train_acc = train_epoch(model, train_loader, criterion, optimizer, device)
        valid_loss, valid_acc = evaluate_model(model, valid_loader, criterion, device)
        elapsed = time.time() - start_time

        train_loss_history.append(train_loss)
        train_acc_history.append(train_acc)
        valid_loss_history.append(valid_loss)
        valid_acc_history.append(valid_acc)

        print(f"  Train loss: {train_loss:.4f}, Train acc: {train_acc:.4f}")
        print(f"  Valid loss: {valid_loss:.4f}, Valid acc: {valid_acc:.4f} - Time: {elapsed:.1
        if valid_acc > best_acc:
            best_acc = valid_acc
    history = {
        'train_loss': train_loss_history,
        'train_acc': train_acc_history,
        'valid_loss': valid_loss_history,
        'valid_acc': valid_acc_history
    }
    return best_acc, history
```

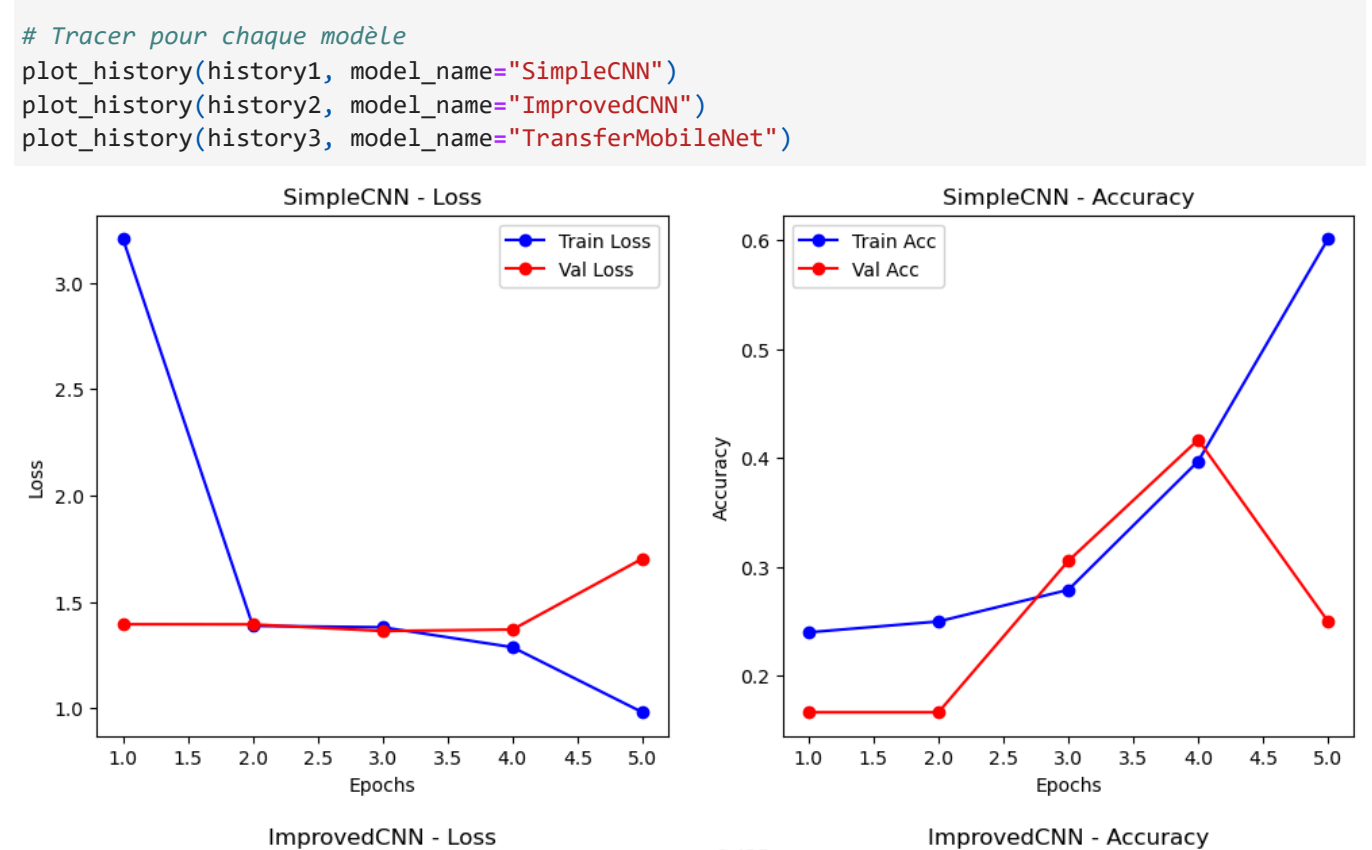
In [12]:

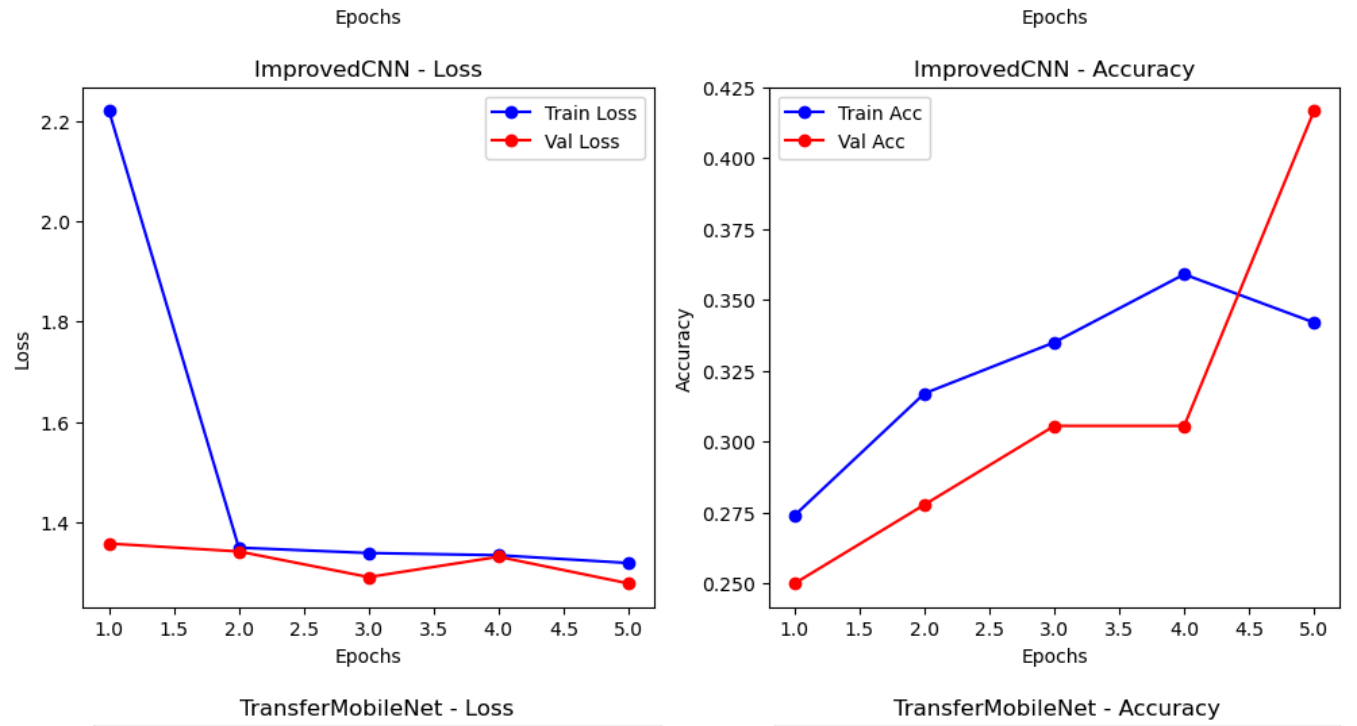
```python
# Configuration du device (GPU si disponible)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("Device utilisé :", device)

num_epochs = 5  # À ajuster selon vos besoins
criterion = nn.CrossEntropyLoss()

# Entraînement du Modèle 1 : SimpleCNN
print("\nEntraînement du Modèle 1 : SimpleCNN")
model1 = SimpleCNN(num_classes=num_classes).to(device)
optimizer1 = optim.Adam(model1.parameters(), lr=0.001)
best_acc1, history1 = train_model(model1, train_loader_basic, valid_loader, criterion, optimiz
                                  num_epochs=num_epochs)

# Entraînement du Modèle 2 : ImprovedCNN
print("\nEntraînement du Modèle 2 : ImprovedCNN")
model2 = ImprovedCNN(num_classes=num_classes).to(device)
optimizer2 = optim.Adam(model2.parameters(), lr=0.0001)
best_acc2, history2 = train_model(model2, train_loader_aug, valid_loader, criterion, optimizer
                                  num_epochs=num_epochs)

# Entraînement du Modèle 3 : TransferMobileNet
print("\nEntraînement du Modèle 3 : TransferMobileNet")
model3 = TransferMobileNet(num_classes=num_classes).to(device)
optimizer3 = optim.Adam(model3.parameters(), lr=0.0001)  # Seul le classificateur est entraîné
best_acc3, history3 = train_model(model3, train_loader_basic, valid_loader, criterion, optimiz
                                  num_epochs=num_epochs)
```

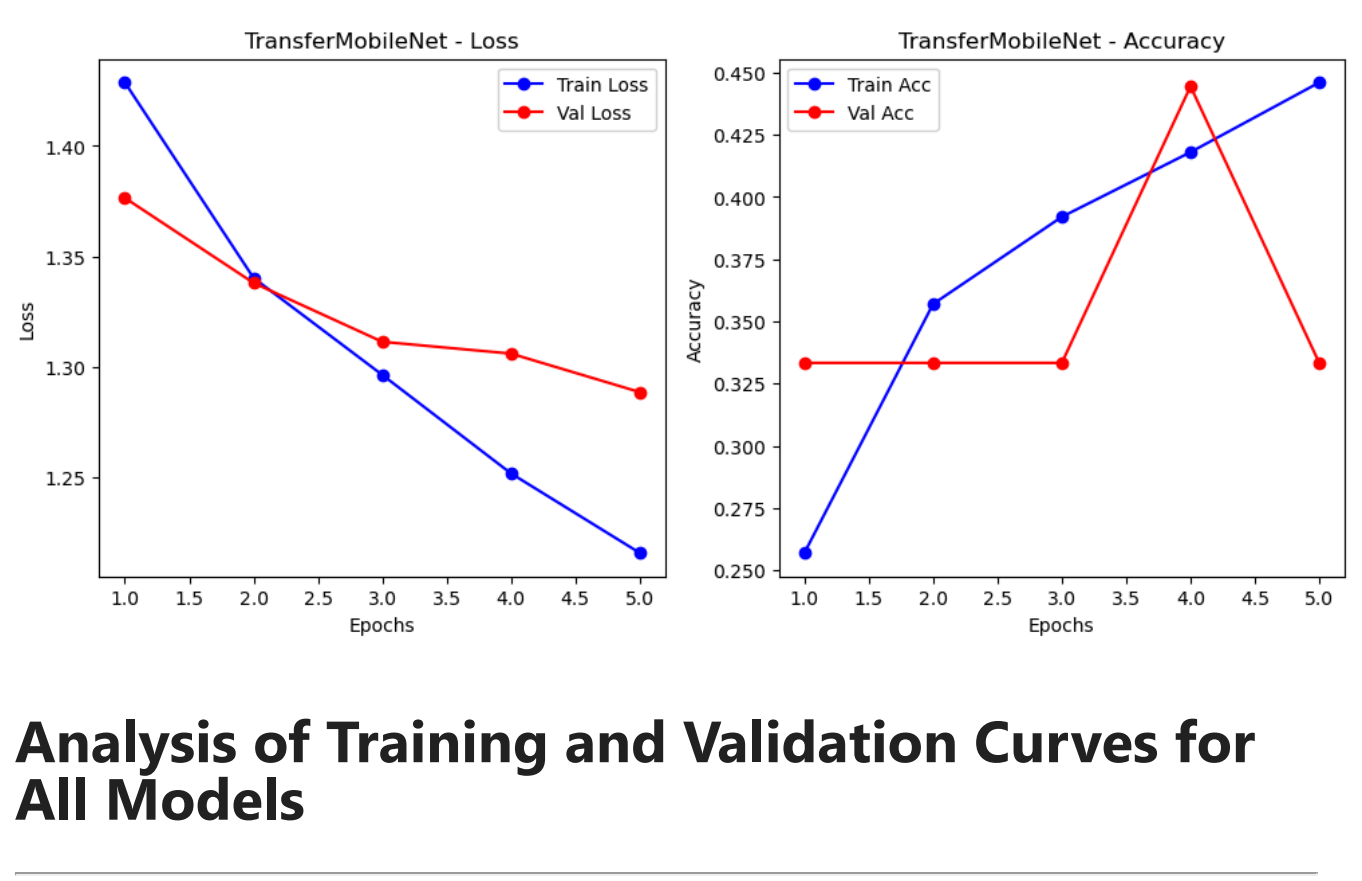
```
Device utilisé : cuda

Entraînement du Modèle 1 : SimpleCNN
Epoch 1/5

  Train loss: 3.2058, Train acc: 0.2400
  Valid loss: 1.3952, Valid acc: 0.1667 - Time: 11.1s

Epoch 2/5

  Train loss: 1.3875, Train acc: 0.2500
  Valid loss: 1.3946, Valid acc: 0.1667 - Time: 10.8s

Epoch 3/5

  Train loss: 1.3810, Train acc: 0.2790
  Valid loss: 1.3633, Valid acc: 0.3056 - Time: 10.4s

Epoch 4/5

  Train loss: 1.2873, Train acc: 0.3970
  Valid loss: 1.3704, Valid acc: 0.4167 - Time: 10.4s

Epoch 5/5

  Train loss: 0.9817, Train acc: 0.6010
  Valid loss: 1.7034, Valid acc: 0.2500 - Time: 10.6s


Entraînement du Modèle 2 : ImprovedCNN
Epoch 1/5

  Train loss: 2.2198, Train acc: 0.2740
  Valid loss: 1.3579, Valid acc: 0.2500 - Time: 11.1s

Epoch 2/5

  Train loss: 1.3497, Train acc: 0.3170
  Valid loss: 1.3421, Valid acc: 0.2778 - Time: 10.9s

Epoch 3/5

  Train loss: 1.3390, Train acc: 0.3350
  Valid loss: 1.2910, Valid acc: 0.3056 - Time: 12.7s

Epoch 4/5

  Train loss: 1.3347, Train acc: 0.3590
  Valid loss: 1.3315, Valid acc: 0.3056 - Time: 13.0s

Epoch 5/5

  Train loss: 1.3192, Train acc: 0.3420
  Valid loss: 1.2785, Valid acc: 0.4167 - Time: 11.7s


Entraînement du Modèle 3 : TransferMobileNet
Epoch 1/5

  Train loss: 1.4287, Train acc: 0.2570
  Valid loss: 1.3765, Valid acc: 0.3333 - Time: 10.4s

Epoch 2/5

  Train loss: 1.3403, Train acc: 0.3570
  Valid loss: 1.3381, Valid acc: 0.3333 - Time: 10.3s

Epoch 3/5

  Train loss: 1.2965, Train acc: 0.3920
  Valid loss: 1.3115, Valid acc: 0.3333 - Time: 10.4s

Epoch 4/5

  Train loss: 1.2520, Train acc: 0.4180
  Valid loss: 1.3061, Valid acc: 0.4444 - Time: 10.3s

Epoch 5/5

  Train loss: 1.2160, Train acc: 0.4460
  Valid loss: 1.2887, Valid acc: 0.3333 - Time: 10.2s
```

In [16]:

```python
def plot_history(history, model_name="Modèle"):
    epochs = range(1, len(history['train_loss']) + 1)

    # Courbe de Loss
    plt.figure(figsize=(12, 5))
    plt.subplot(1, 2, 1)
    plt.plot(epochs, history['train_loss'], 'bo-', label='Train Loss')
    plt.plot(epochs, history['valid_loss'], 'ro-', label='Val Loss')
    plt.title(f"{model_name} - Loss")
    plt.xlabel("Epochs")
    plt.ylabel("Loss")
    plt.legend()

    # Courbe d'Accuracy
    plt.subplot(1, 2, 2)
    plt.plot(epochs, history['train_acc'], 'bo-', label='Train Acc')
    plt.plot(epochs, history['valid_acc'], 'ro-', label='Val Acc')
    plt.title(f"{model_name} - Accuracy")
    plt.xlabel("Epochs")
    plt.ylabel("Accuracy")
    plt.legend()

    plt.show()


# Tracer pour chaque modèle
plot_history(history1, model_name="SimpleCNN")
plot_history(history2, model_name="ImprovedCNN")
plot_history(history3, model_name="TransferMobileNet")
```







# Analysis of Training and Validation Curves for All Models

---

## 1. SimpleCNN Analysis

### Loss Curve (Left Graph)

#### Observation:

- The **training loss (blue line)** drops significantly between **epochs 1 and 2**, indicating rapid learning.
- The **validation loss (red line)** remains relatively stable but starts increasing after **epoch 4**.
- A noticeable gap between training and validation loss appears after **epoch 4**, suggesting **overfitting**.

#### Interpretation:

- The model initially **generalizes well** but starts **overfitting** after epoch 4.
- The increasing validation loss suggests that the model is **memorizing** the training data rather than learning general features.
- Possible solutions include adding **dropout layers** or **early stopping**.

### Accuracy Curve (Right Graph)

#### Observation:

- The **training accuracy (blue line)** increases steadily.
- The **validation accuracy (red line)** follows a similar trend but drops at **epoch 5**.
- A large divergence appears at **epoch 5**, reinforcing the **overfitting problem**.

#### Interpretation:

- The model is **performing well on training data** but struggles with generalization.
- The sharp drop in validation accuracy suggests that the model may require **regularization techniques** like dropout or weight decay.

---

## 2. ImprovedCNN Analysis

### Loss Curve (Left Graph)

#### Observation:

- The **training loss (blue line)** decreases significantly at the start and then stabilizes.
- The **validation loss (red line)** follows a similar pattern but remains close to the training loss.
- The **loss curves stay close together**, suggesting **better generalization**.

#### Interpretation:

- This model appears to **generalize better** than SimpleCNN.
- The **Batch Normalization and Dropout** layers help in reducing overfitting.
- The **stability** in validation loss suggests that the model is **learning meaningful features**.

### Accuracy Curve (Right Graph)

#### Observation:

- Both **training and validation accuracy** improve steadily over epochs.
- The validation accuracy **remains close to the training accuracy**, showing **better generalization** compared to SimpleCNN.
- At **epoch 5**, validation accuracy jumps significantly.

#### Interpretation:

- The **ImprovedCNN model is more stable**, reducing overfitting.
- The steady increase in validation accuracy suggests that **data augmentation and batch normalization are beneficial**.
- The final jump in validation accuracy at **epoch 5** suggests that the model is learning more useful patterns.

---

## 3. TransferMobileNet Analysis

### Loss Curve (Left Graph)

#### Observation:

- Both **training and validation loss** decrease consistently.
- The curves remain **close together**, suggesting **low overfitting**.
- The validation loss stabilizes at **epoch 3**, while the training loss keeps decreasing.

#### Interpretation:

- **Transfer learning is effective** since MobileNetV2 was **pretrained on ImageNet**.
- The **small gap** between training and validation loss indicates **good generalization**.
- **Fine-tuning more layers in MobileNetV2** could further improve performance.

### Accuracy Curve (Right Graph)

#### Observation:

- Training accuracy **steadily increases**.
- Validation accuracy **fluctuates**, peaking at **epoch 4** and then dropping slightly.
- The gap between training and validation accuracy **remains moderate**.

#### Interpretation:

- The peak at **epoch 4** followed by a drop suggests **overfitting at later epochs**.
- The model still performs **better than SimpleCNN and ImprovedCNN** in terms of **generalization**.
- **Fine-tuning additional layers in MobileNetV2** could further improve performance.

In [20]:

```python
test_loss1, test_acc1 = evaluate_model(model1, test_loader, criterion, device)

test_loss2, test_acc2 = evaluate_model(model2, test_loader, criterion, device)

test_loss3, test_acc3 = evaluate_model(model3, test_loader, criterion, device)

# Sélection du meilleur modèle
test_accuracies = [test_acc1, test_acc2, test_acc3]
best_model_index = np.argmax(test_accuracies)
model_names = ["1 (CNN classic)", "2 (CNN improved)", "3 (Learning transfer)"]

rapport = f"""
Training of 3 models:
   - Model 1: Basic CNN (SimpleCNN).
   - Model 2: Improved CNN.
   - Model 3: Transfer of learning with MobileNetV2 (TransferMobileNet).
   - Each model was trained for {num_epochs} epochs.

   - Modèle 1 : Accuracy = {test_acc1:.4f}
   - Modèle 2 : Accuracy = {test_acc2:.4f}
   - Modèle 3 : Accuracy = {test_acc3:.4f}

    {model_names[best_model_index]} is recommended for its performance and robustness.

"""
print(rapport)
```

```
Training of 3 models:
   - Model 1: Basic CNN (SimpleCNN).
   - Model 2: Improved CNN.
   - Model 3: Transfer of learning with MobileNetV2 (TransferMobileNet).
   - Each model was trained for 5 epochs.

   - Modèle 1 : Accuracy = 0.3684
   - Modèle 2 : Accuracy = 0.2368
   - Modèle 3 : Accuracy = 0.4211

    3 (Learning transfer) is recommended for its performance and robustness.
```

---

## Final Comparison and Conclusion

| Model | Overfitting | Stability | Best Generalization | Accuracy Trend |
|---|---|---|---|---|
| **SimpleCNN** | High | Poor | Weak | Overfits after epoch 4 |
| **ImprovedCNN** | Moderate | Good | Decent | Improves steadily |
| **TransferMobileNet** | Low | Best | Strong | Some fluctuations but best generalization |

### Key Findings:

- **Best Model: TransferMobileNet** → Shows the **best generalization** with the lowest overfitting.
- **Most Stable Model: ImprovedCNN** → Regularization techniques (BatchNorm, Dropout) helped.
- **Worst Model for Generalization: SimpleCNN** → High overfitting and unstable validation performance.

---

## Suggested Next Steps

1. **Fine-tune TransferMobileNet** by unfreezing some layers.
2. **Increase training epochs** for ImprovedCNN to see if it catches up.
3. **Use dropout & weight decay** to reduce overfitting in SimpleCNN.
4. **Experiment with different architectures** like EfficientNet or ResNet50.

### Final Recommendation

**TransferMobileNet** is the best-performing model, but additional **fine-tuning** could further improve its stability and accuracy.